Lecture 2: Processes and Threads

Process Concept

A process is a program in execution. It includes the program code (text section), current activity (program counter, registers), stack (temporary data), data section (global variables), and heap (dynamically allocated memory).

Process states: New, Ready, Running, Waiting, Terminated.

Process Control Block (PCB)

The PCB contains all information about a process: process state, program counter, CPU registers, CPU scheduling information, memory management info, I/O status, and accounting information.

The PCB is the kernel's representation of a process. Context switches save/restore PCB contents.

Threads

A thread is the basic unit of CPU utilization. It comprises a thread ID, program counter, register set, and stack. Threads within the same process share code, data, and OS resources.

Benefits of multithreading: responsiveness, resource sharing, economy (cheaper than creating new processes), and scalability on multiprocessor systems.

Lecture 2 (cont.): Scheduling

CPU Scheduling Algorithms

First-Come, First-Served (FCFS): Simple but can cause the convoy effect where short processes wait behind long ones. Non-preemptive.

Shortest Job First (SJF): Optimal for minimizing average waiting time, but requires knowledge of future burst lengths. Can be preemptive (SRTF) or non-preemptive.

Round Robin (RR): Each process gets a small time quantum (10-100ms). After the quantum expires, the process is preempted and moved to the end of the ready queue. Good for time-sharing systems.

Priority Scheduling: Each process has a priority. Higher priority processes run first. Can suffer from starvation of low-priority processes. Solution: aging.

Context Switching

A context switch saves the state of the old process and loads the saved state of the new process. This is pure overhead (no useful work done). Typical cost: 1-10 microseconds on modern hardware.